

Yazılım Mühendisliğine Giriş

Kodlamanın Ötesi: Sistemleri, Süreçleri ve Geleceği Tasarlamak



Sadece kod yazan biri olmaktan, küresel standartlarda sistemler inşa eden bir mühendise dönüşme yolculuğuna hoş geldiniz.

Bu Derste Neler Kazanacaksınız?



Büyük Resmi Görmek

Yazılım mühendisliği sürecini analizden bakıma kadar uçtan uca anlama ve yönetme yetkinliği.

Standartları Uygulamak

SWEBOK, IEEE gibi uluslararası normlarda güvenli kod ve mimari geliştirme pratiği.

Mühendislik Yaklaşımı

Sadece problemi çözmek değil; zaman, maliyet, performans ve etik kısıtları bir arada optimize etmek.

Takım ve Proje Ruhu

Modüler yapıları inceleme, konfigürasyon yönetimi ve gerçek dünya problemlerine takım olarak çözüm üretme.

Bir Yanılgıyı Düzeltelim: Yazılım Sadece 'Kod' Değildir

Anlamalı bir ürün ortaya koymak için 4 temel bileşenin entegrasyonu gerekir.



1. Mantık (Algoritma)

Yazılımın beyni. Elektronik ortama taşınmak istenen işin kuralları ve çözüm yolları. (En kritik bileşen).



2. İnsan (Kullanıcı ve Üretici)

Yazılımı tasarlayan çok disiplinli ekipler ve onu kullanacak olan son kullanıcılar.



3. Belge (Dokümantasyon)

Planlama, çözümleme, tasarım ve test aşamalarında üretilen kurumsal hafıza.



4. Program (Kodlar)

Donanıma hayat veren, işi icra eden makine diline çevrilmiş yapısal ifadeler.

"Yazılım Krizi"nden Modern Mühendisliğe

1968 (Kıvılcım) 

NATO Konferansı.
Yazılımın karmaşıklaşmasıyla ortaya çıkan krize karşı

"Mühendislik" prensiplerinin gerekliliğinin ilk kez dünya çapında kabulü.

1980'ler 

Sınırlı dağıtım, kişiye/kuruma özel müşteri yazılımları ve temel veritabanları çağı.

1990'lar 

Çoklu kullanıcı, gerçek zamanlı sistemler ve **"Nesneye Yönelik (OO)"** teknolojilerin yükselişi.

2000'ler ve Ötesi 

Yapay sinir ağları, paralel işleme (**Grid Computing**), gömülü sistemler ve düşük maliyetli donanımların getirdiği devasa tüketici etkisi.

Mühendis Kimliği: Sizi Diğerlerinden Ne Ayıracak?

Geleneksel Programcı

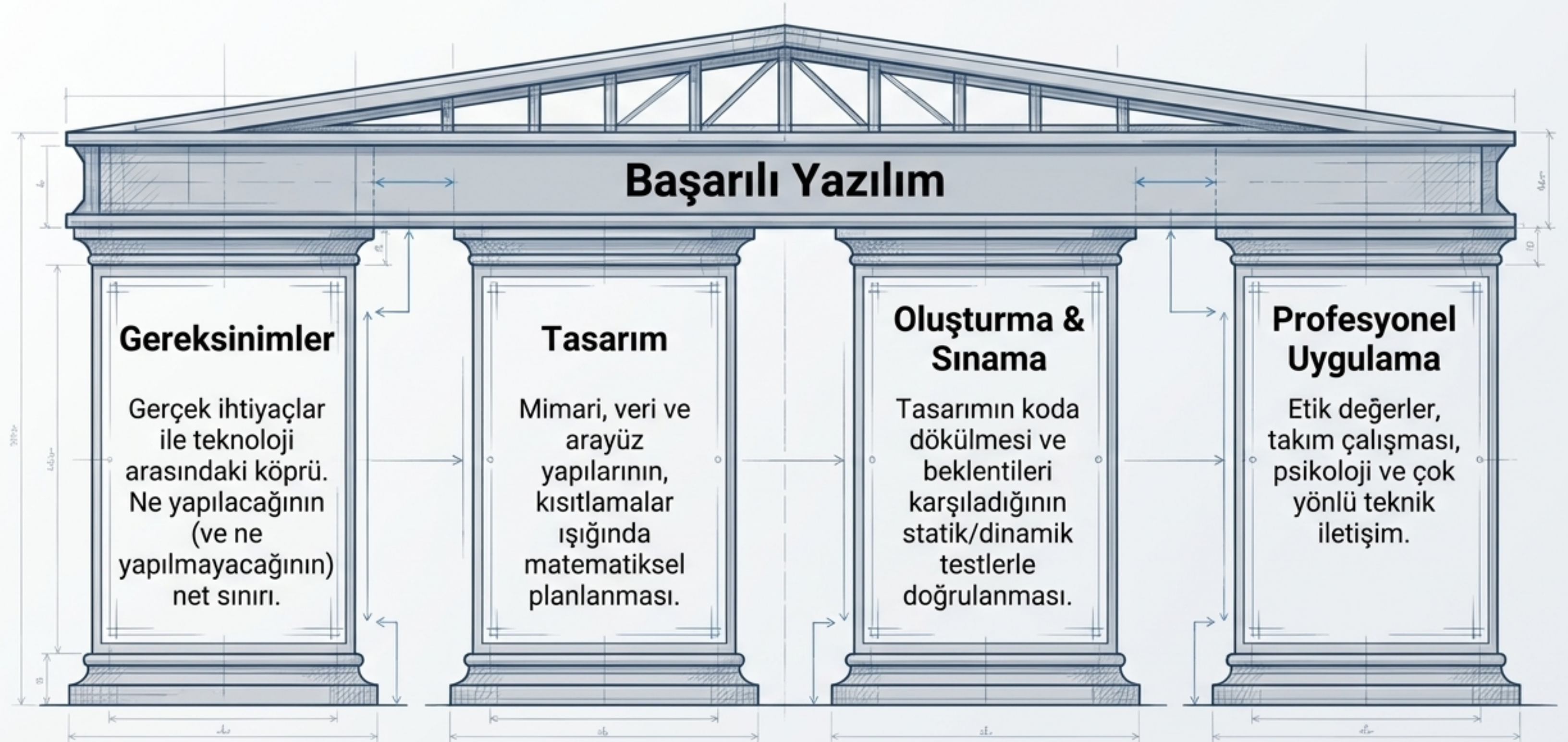
Odak noktası sadece kod yazmak ve anlık problemi çözmektir. Genellikle izole ve bağımsız çalışmaya meyillidir.

Yazılım Mühendisi

- Programcının yeteneklerine sahiptir ancak parçaları **bütünleştirici (entegratör)** olarak çalışır.
- **Maliyet, zaman, organizasyon** ve **etik** kısıtlamalar içinde **optimum çözümü** arar.
- Kullanıcı gereksinimlerini **teknik mimariye** çeviren bir köprüdür.
- **Disiplinli, tekrarlanabilir ve öngörülebilir bir süreç yönetir.**

Mesleğimizin Temel Taşları (SWEBOK)

IEEE tarafından belirlenen "Yazılım Mühendisliği Çekirdek Bilgi Alanları"



Bizim İçin "İyi Yazılım" Ne Demektir?

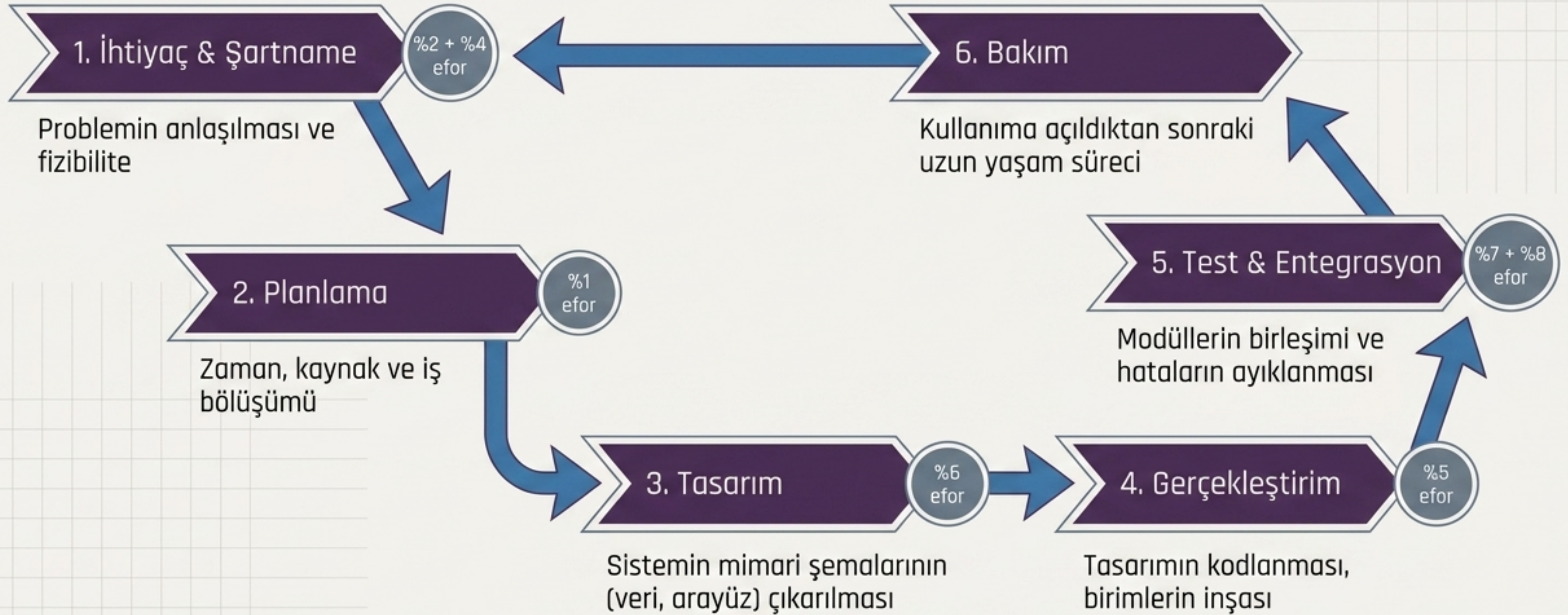
Sürecin ve ürünün kalitesi ayrılamaz. İyi yazılım sadece çalışan değil, "doğru ve güvenli" çalışan yazılımdır.



Etik Vurgusu (SWCEPP Kodu):

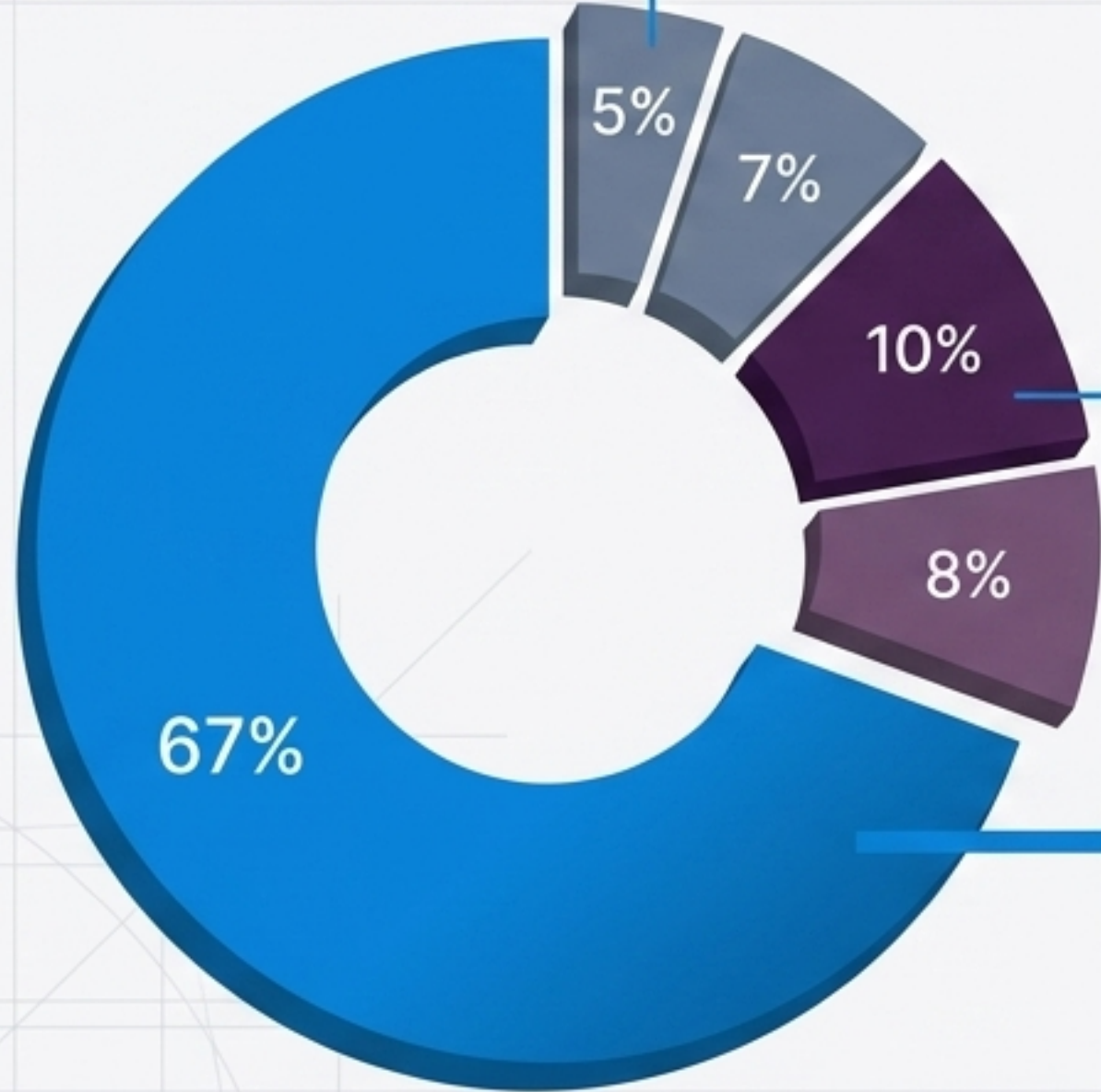
Bir mühendis; sosyal, yasal ve ekonomik çıkarları bütünleştirmeli, mesleki ahlakı (hukuki kavramlar, gizlilik, toplumsal fayda) her şeyin üstünde tutmalıdır.

Yazılım Yaşam Döngüsü (SDLC) Aşamaları



Sektördeki En Büyük Yanılgı: "Bakım" Gerçeği

"Yazılım bittiğinde işimiz bitmez, asıl iş o zaman başlar."



Sadece %5 Kodlama

Gerçekleştirim tüm sürecin yalnızca %5'ini kapsar!

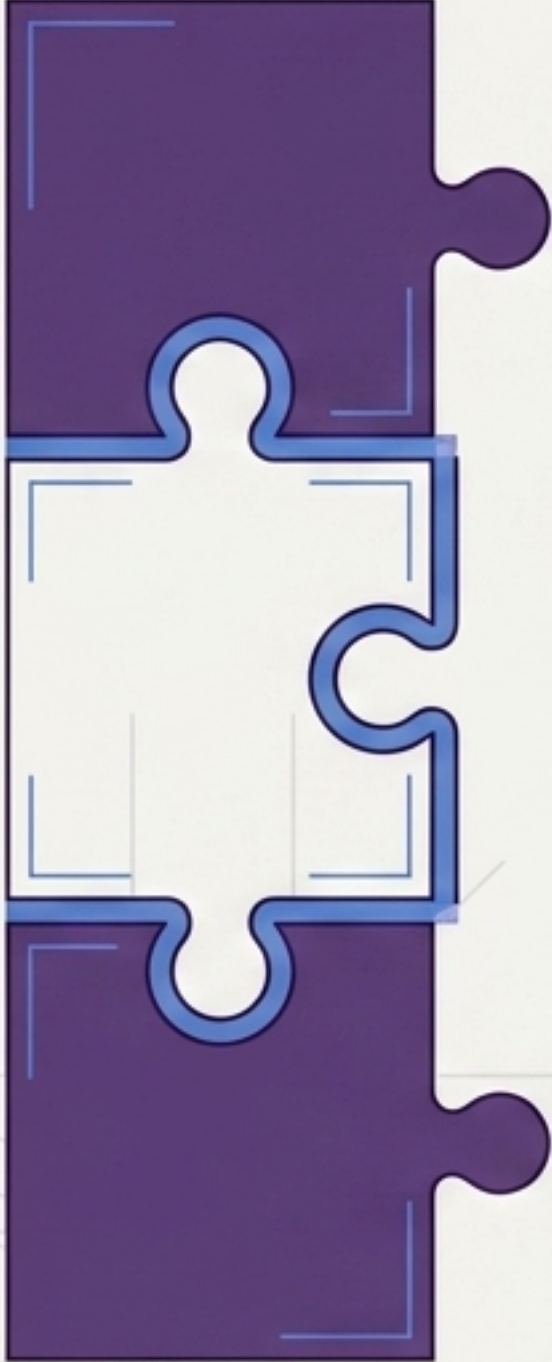
%15 Test ve Bütünleme

Hataları bulmak ve sistemleri birleştirmek, kodlamadan 3 kat daha fazla efor gerektirir.

%67 BAKIM

Dinamik işletme yapıları, sürekli değişen ihtiyaçlar ve giderilemeyen hatalar nedeniyle bütçenin ve zamanın 2/3'ü yazılım canlıya alındıktan sonra harcanır. İyi mühendislik bu %67'yi yönetebilme sanatıdır.

Sınıftan Gerçek Dünyaya: Eğitim Modelimiz



Temel Bilimler

Algoritmalar, veritabanları, ağ sistemleri (Bilgisayar Bilimi Temelleri).

Ekip Projesi

Gerçek bir konfigürasyon yönetimi ve takım ruhu için dönemlik proje. (Tek başına kod yazmak yok!)

Araştırma ve Raporlama

Bize hazır verilmeyen bilgiyi arama, literatür tarama ve profesyonel belge üretme alışkanlığı.

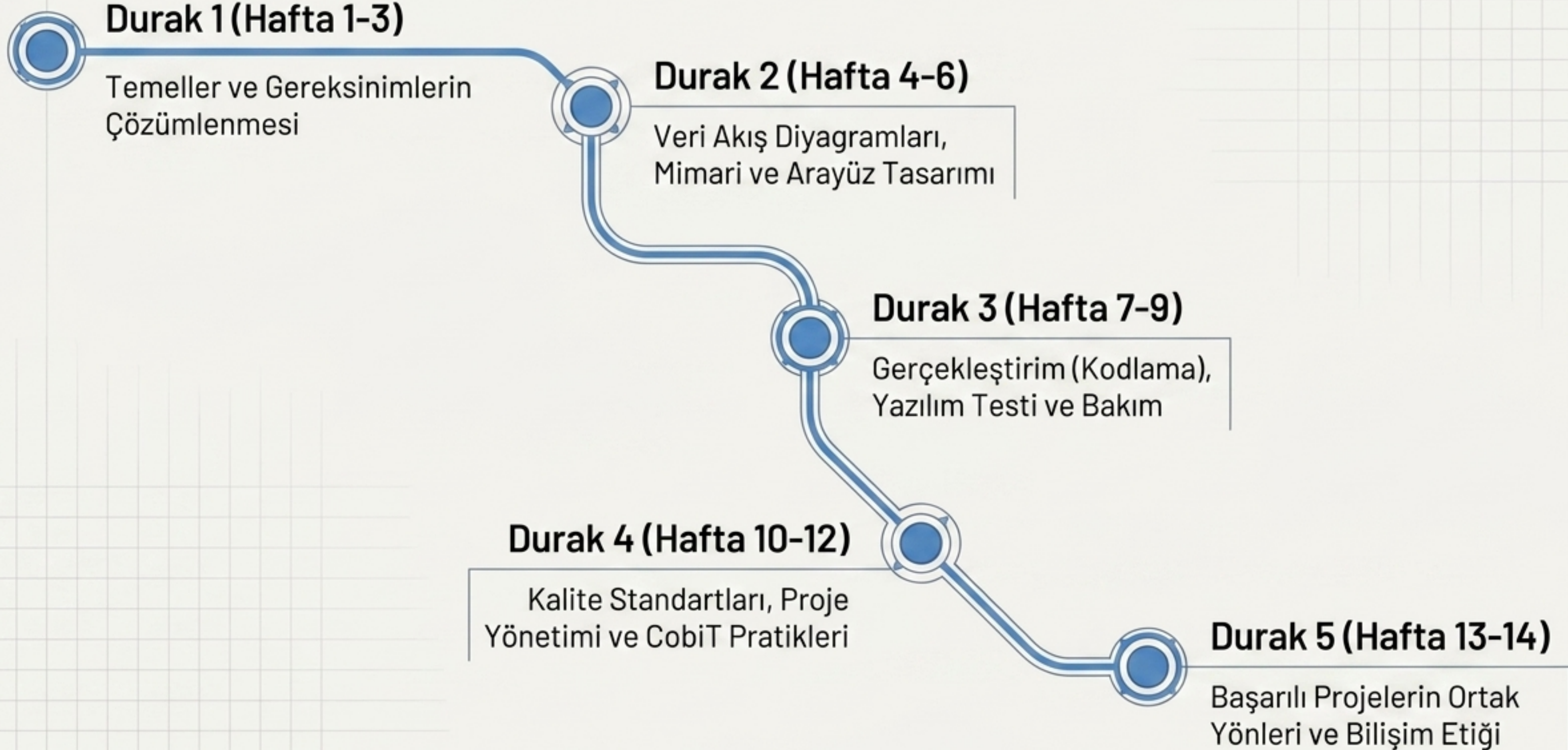
Çok Yönlü Değerlendirme (Puanlama)

Yıliçi Başarısı: %50

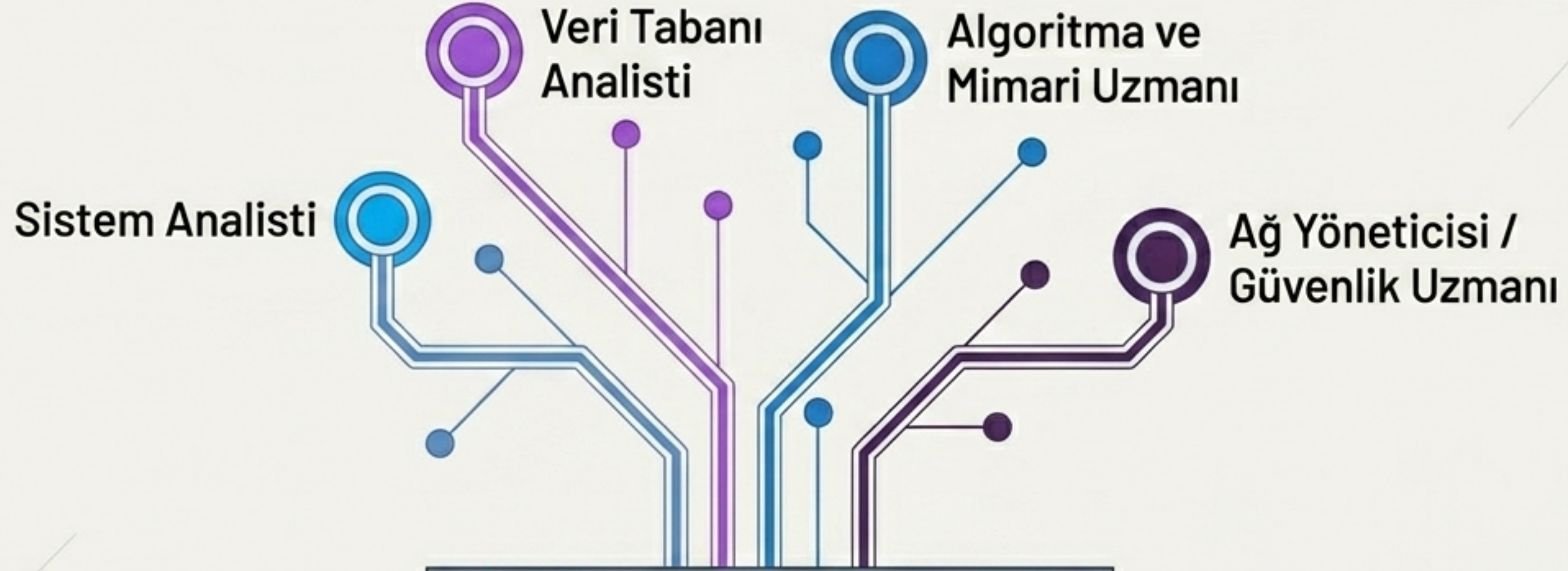
Ara Sınav: %70, Kısa Sınavlar: %20, Ödev: %10

Final Başarısı: %50

14 Haftalık Keşif Yolculuğumuz



Mezun Olduğunuzda Hangi Şapkayı Takacaksınız?



Temel Yazılımcılık (İlk 4 Dönem)

- Algoritma, Veri Tabanı Analizi,
- Programlama Dilleri, Sistem Analizi.

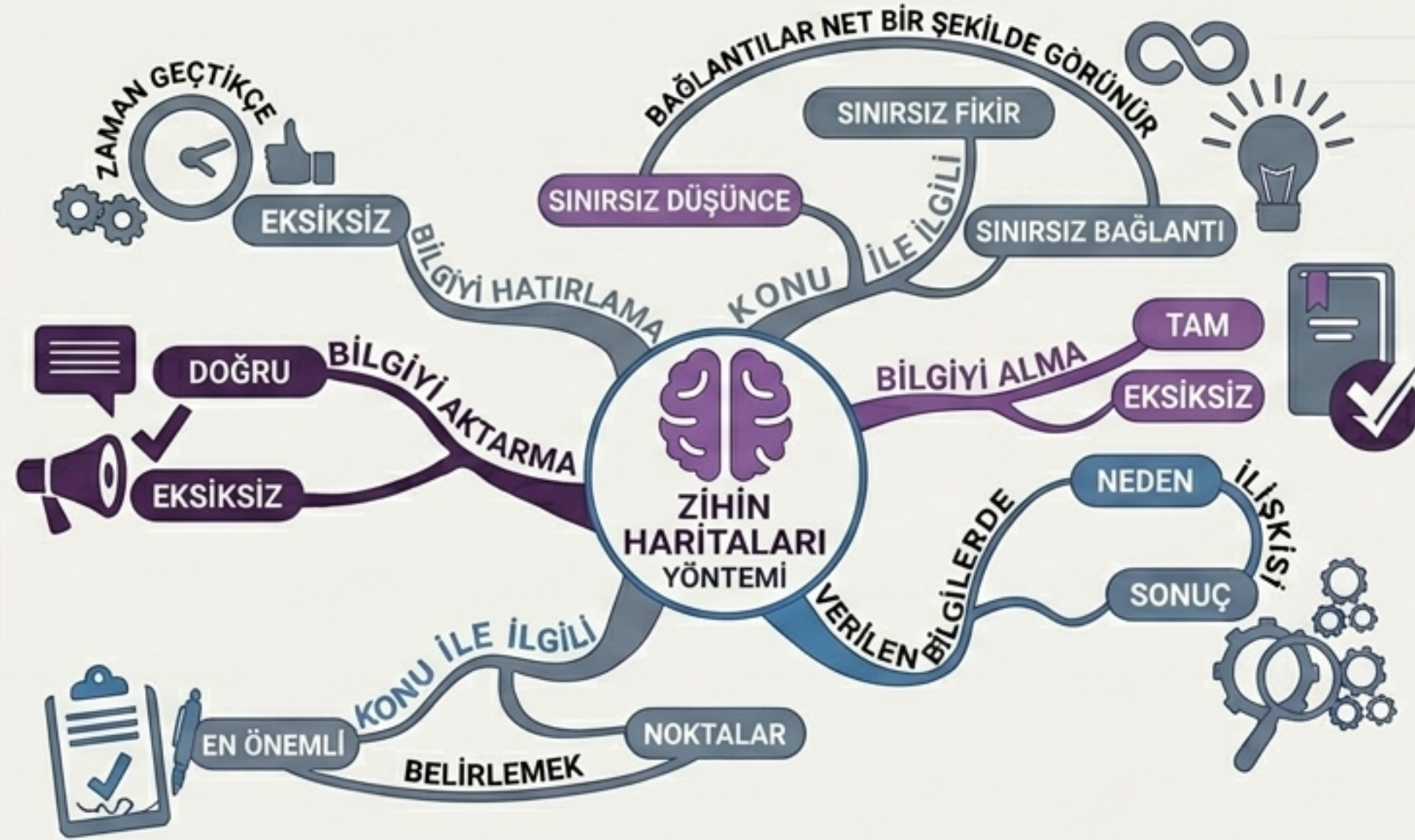
Uzmanlaştıkça üretilen işin kalitesi artar.

Karmaşık Sistemleri Nasıl Çözeceğiz?

Soyut kavramları somutlaştırmak için kullanacağımız bir numaralı teknik.

Araç

Gereksinimleri analiz ederken ve mimariyi kurarken Zihin Haritaları (Mind Mapping) kullanacağız.



İşlev

Bilgiyi eksiksiz alma, neden-sonuç ilişkilerini ağ yapısında kurma ve sınırsız düşünceyi görselleştirme.

Mühendislik Faydası

Kod yazmaya başlamadan önce, projenin sınırlarını ve bağlantılarını kağıt üzerinde limitsizce modellemenin en etkili yoludur.



Yolculuk Başlıyor...

Bugün, sadece kod satırlarına değil, insan hayatına dokunan sistemler tasarlamak üzere ilk adımı attınız. Mühendislik, bir problemi çözmek kadar, o çözümü başkaları için sürdürülebilir kılma sanatıdır.

Haftaya Hazırlık (Görev):

- SWEBOK kavramlarını ve yazılım geliştirme modellerini inceleyerek geliniz.
- Okuma Önerisi: R. Pressman - Software Engineering: A Practitioner's Approach.